AIMERZ.ai
Aim.Act.Achieve

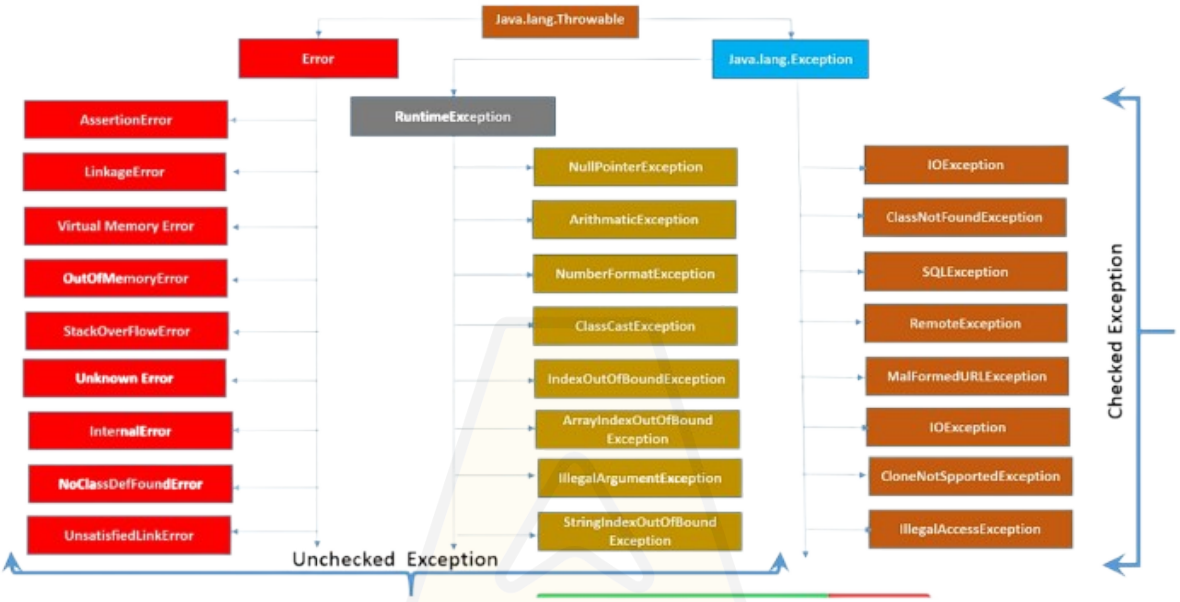
NOTES

Handling Exceptions in Java -I

Exception Handling in Java
Error Vs Exception
Exception Handling using try-catch
using Finally Keyword-Block

Exception Handling in Java

Exception handling in Java is essential for writing robust programs that handle unexpected behaviors smoothly. Instead of crashing the program when an error occurs, exception handling provides a way to deal with it gracefully.



Now let's begin with **Error Vs Exception**

What is an Error? Errors represent severe problems that typically occur at the system or environment level. These are generally issues outside the control of the program, and Java programs are not expected to recover from errors. Errors usually indicate fundamental problems in the runtime environment, such as resource exhaustion, system crashes, or serious configuration issues.

Real-World Analogy: Irrecoverable Failures : An error is like the structural failure of a building foundation. Just as such a failure cannot be fixed without rebuilding the foundation, certain errors in a program require restarting the JVM or addressing hardware limitations.

Characteristics of Errors:

Errors are usually unrecoverable, meaning they cannot be handled within the program.

They occur at the system level, often due to limitations of hardware or the environment (like JVM running out of memory).

Java provides the Error class for errors, and it includes several subclasses like OutOfMemoryError and StackOverflowError.

Example of Error: StackOverflowError

A StackOverflowError occurs when a method calls itself recursively without a base case, causing an infinite recursion that exhausts the program's stack memory.

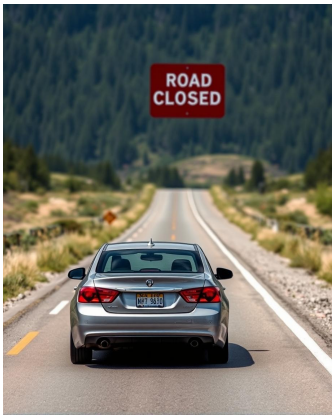
```
Java
public class StackOverflowExample {
    public static void recursiveMethod() {
        recursiveMethod(); // Infinite recursion
    }

    public static void main(String[] args) {
        recursiveMethod(); // Triggers StackOverflowError
    }
}
```

What is an Exception?

An *exception* is an event that disrupts the normal flow of a program. In Java, exceptions are objects representing errors or unexpected conditions.

Real-World Analogy: Roadblocks

Imagine driving on a road and suddenly seeing a "Road Closed" sign. Instead of stopping there, you might take a detour. Similarly, exceptions in Java help the program take a different path when something goes wrong.



Let's begin by understanding Java's Exception Handling Mechanism

Java provides keywords to handle exceptions:

- **try**: Defines a block of code that might throw an exception.
- **catch**: Used to catch and handle exceptions thrown in the try block.
- **finally**: Executes code regardless of whether an exception occurs, often for cleanup.
- **throw**: Used to manually throw an exception.
- **throws**: Declares that a method may throw specific exceptions.

Exception Handling using try-catch Blocks

The try-catch block is used to handle exceptions. Code that might throw an exception goes in the try block. If an exception occurs, control transfers to the catch block.

Example Syntax

```java
try {
    // code that might throw an exception
} catch (ExceptionType e) {
    // code to handle the exception
}
```

Example: Dividing by Zero

```java
public class DivideExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // Causes an
ArithmeticException
```

```
        } catch (ArithmeticException e) {
            System.out.println("You cannot divide by
zero!");
        }
    }
}
```

Real-World Analogy: Umbrella on a Rainy Day: Using try-catch is like carrying an umbrella on a rainy day. If it rains (exception), you open the umbrella (catch block) to stay dry.

Uncaught Exceptions and Default Exception Handler

When an exception is unhandled, Java's runtime system uses a default exception handler to:

Print a description of the exception.
Show a stack trace to the exception's origin in the code and
Terminate the program.

Example Code

```Java
public class UncaughtExceptionExample {
    public static void main(String[] args) {
        int num = 10 / 0; // Causes an
ArithmeticException
        System.out.println("This line will not be
printed.");
    }
}
```

Output:

```
Java
Exception in thread "main" java.lang.ArithmeticException:
/ by zero
at
UncaughtExceptionExample.main(UncaughtExceptionExample.ja
va:3)
```

Using Finally Block

The finally block always executes after the try and catch blocks, regardless of whether an exception occurred. This is ideal for cleanup tasks like closing files or releasing resources.

Real-World Analogy: Locking the Door : Leaving the house requires locking the door whether you remembered everything (try-catch) or not (finally).

Syntax

```
Java
try {
    // code that might throw an exception
} catch (ExceptionType e) {
    // handle the exception
} finally {
    // code that will always execute
}
```

Example: Cleaning Up Resources

```
Java
Scanner scanner = null;
```

```java
try {
    scanner = new Scanner("file.txt");
    // Read data from file
} catch (IOException e) {
    System.out.println("File not found or cannot be
opened.");
} finally {
    if (file ≠ null) {
        try {
            scanner.close();
        } catch (IOException e) {
            System.out.println("Error closing the
file.");
        }
    }
}
```

Key Points

finally block: Always executes, regardless of whether an exception is thrown or caught.

Common Use Cases for finally:

While the previous example was simplified, here are some common use cases for the finally block:

Closing Files: Ensure that files are closed properly to prevent resource leaks.

Releasing Database Connections: Release database connections to avoid connection pool exhaustion.

Logging Error Messages: Log error messages for debugging and analysis, even if an exception occurs.

Committing or Rolling Back Transactions: Ensure that database transactions are either committed or rolled back.

Types of Exceptions

Checked Exceptions: Exceptions checked at compile-time, which Java requires you to handle or specify in the method signature. Example: IOException.

Unchecked Exceptions: These occur at runtime, and Java doesn't require you to handle them. Example: ArithmeticException, like dividing by zero.

Errors: Errors are serious issues outside the application's control, typically related to system resources, such as StackOverflowError or OutOfMemoryError.

Real-World Analogy: Expected and Unexpected Events

If you're planning a trip, you prepare for expected road closures (checked exceptions). But if your car suddenly breaks down (unchecked exception), you need to handle it on the spot.

Now let's explore, Types of Exception in Java

Types of Exceptions : Java exceptions are divided into two main categories:

- ➔ **Checked Exceptions**

➔ **Unchecked Exceptions**

Let's begin with Checked Exceptions:
Checked exceptions are the exceptions that the *compiler checks* at compile time. These exceptions must be handled using try-catch blocks or declared using the throws keyword. If they're not handled, the program won't compile.

Analogy:
Imagine you're boarding a flight. Before you enter the plane, airport security checks your boarding pass. If you don't have it, you're not allowed to board. Similarly, Java ensures that checked exceptions are addressed before the program runs.

Common Examples:

➔ IOException
➔ SQLException
➔ FileNotFoundException

Now let's see Unchecked Exceptions:

Unchecked exceptions are the exceptions that the *compiler does not check* at compile time. These exceptions occur due to programming mistakes and are not required to be explicitly handled.

Analogy:
Think of unchecked exceptions like tripping over a crack in the sidewalk. No one checks the ground for cracks in advance—it's up to you to be careful while walking.

Common Examples:

➔ NullPointerException
➔ ArrayIndexOutOfBoundsException
➔ ArithmeticException

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan